FEU INSTITUTE OF TECHNOLOGY

IT0093
(MOBILE APPLICATION DEVELOPMENT 2)

Course Project

Student Organizer

"Student Organizer" — Navigation, Lists, Core Data, Photos, and Collection Views

Student Name / Group Name:	
Members (if Group):	**Name** / **Role**
Section:	
Professor:	

1. Coverage

This Course Project is a graded, individually-submitted deliverable that consolidates the lessons of the Laboratory Manual on **Navigation and User Interaction**, **Lists and Displays**, **Data Persistence**, **Images and Photo Picker**, and **UICollectionView**.

Unlike the guided in-class activities, this project asks you to build one complete, independently-scoped application, **Student Organizer**, from a written specification and the step-by-step guide in Section 5. The interface is designed in **Storyboard** using **UIKit**, and all data is stored locally on the device with **Core Data**. The project must be created from a new Xcode template following the steps below; copying another student's project files and renaming classes does not satisfy this requirement.

Topic	Concept	Where it appears in this Course Project
Navigation and User Interaction	UINavigationController, manual Show segues with identifiers, performSegue(withIdentifier:sender:), prepare(for:sender:), bar button items and IBAction, UIAlertController, UITextField, UITextView, UISwitch, UISegmentedControl, UIDatePicker	Part B, Steps 14–20; Part C, Steps 21–30; Part F, Steps 43–47
Lists and Displays	UITableViewController, prototype cells, data source and delegate methods, subtitle cells, swipe actions, empty-state labels	Part B, Steps 14–20; Part C, Steps 21–30
Data Persistence	Core Data model (entities, attributes, relationships, delete rules), NSPersistentContainer, NSManagedObjectContext, NSFetchRequest, NSPredicate, save and delete	Part A, Steps 1–13 (model); used in Parts B–F
Images and Photo Picker	PHPickerViewController, NSItemProvider, UIImage resizing, JPEG data stored in a Binary Data attribute	Part E, Steps 38–42
UICollectionView	Flow layout, custom UICollectionViewCell, UICollectionViewDataSource, UICollectionViewDelegateFlowLayout, didSelectItemAt	Part D, Steps 31–37
Professional Practice	Git version control, testing, code documentation, running the app on a physical iPhone	Part A, Step 2; Part G, Steps 48–51; Part H, Steps 52–62

2. CLO Alignment

- **CLO 1** — Apply Swift programming concepts and UIKit components (UIViewController, UITableView, UICollectionView, UILabel, UIImageView, UITextField, UISwitch, UISegmentedControl, UIDatePicker, IBOutlet, IBAction).
- **CLO 2** — Design a Storyboard-based iOS application that incorporates a multi-screen user interface and local data persistence with Core Data satisfying a written specification.
- **CLO 3** — Follow professional development practices: version control from project creation, descriptive naming, defensive handling of optional user input, testing on a physical device, and a written reflection.

3. Scenario

Student Organizer is a lightweight companion app that helps a student keep track of schoolwork on their iPhone. The student creates **subjects** (for example “Web Systems”). Inside each subject they keep a list of **tasks** with a due date, a priority, and a done/not-done status, and a **photo notes board** where each note has text and an optional photo, for example a picture of a whiteboard. Everything is saved on the device with Core Data, so the information is still there the next time the app is opened. The app needs no internet connection.

You are free to choose your own subject names, accent colors, and SF Symbols icons, provided every functional requirement in Section 5 is met.

4. Expected Output / Outcomes

The mockups below illustrate the target outcome described in Section 3 and the requirements in Section 5. They are a visual reference, not a required color scheme or layout. Your own colors and icons may differ.

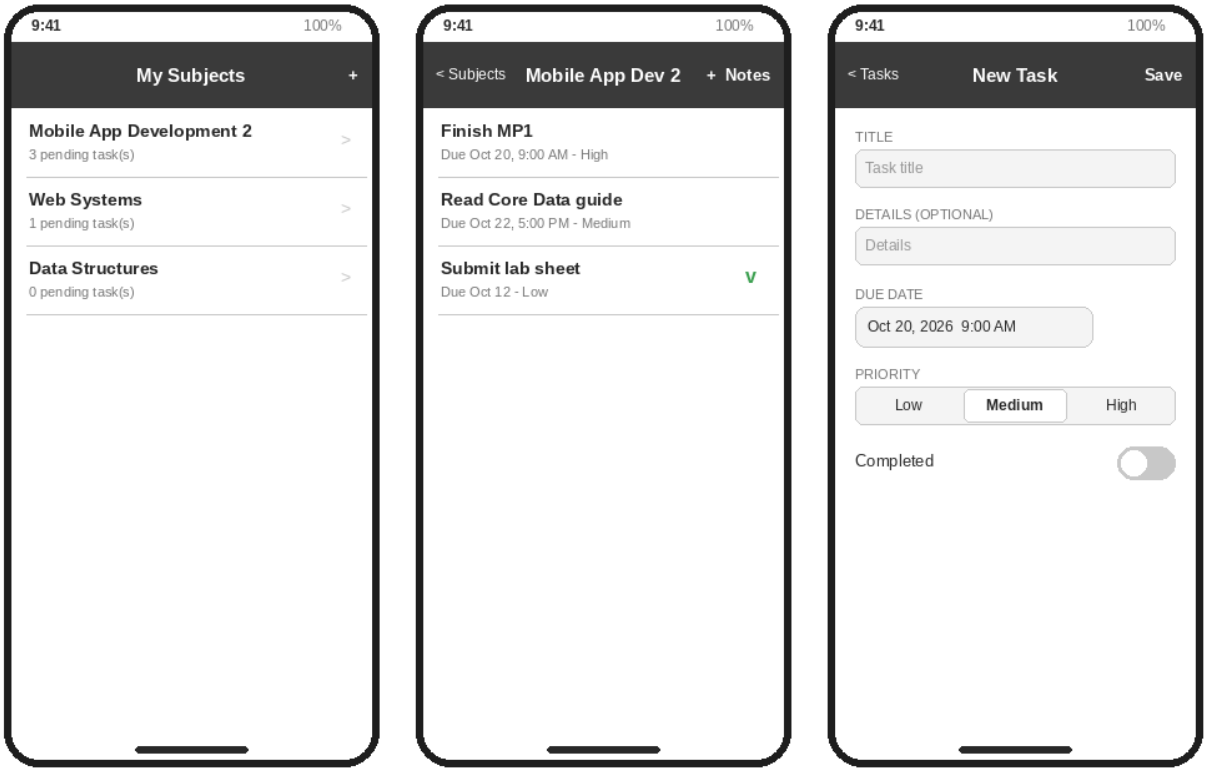


Figure 1. Subjects list, Tasks list, and Task Form screens.

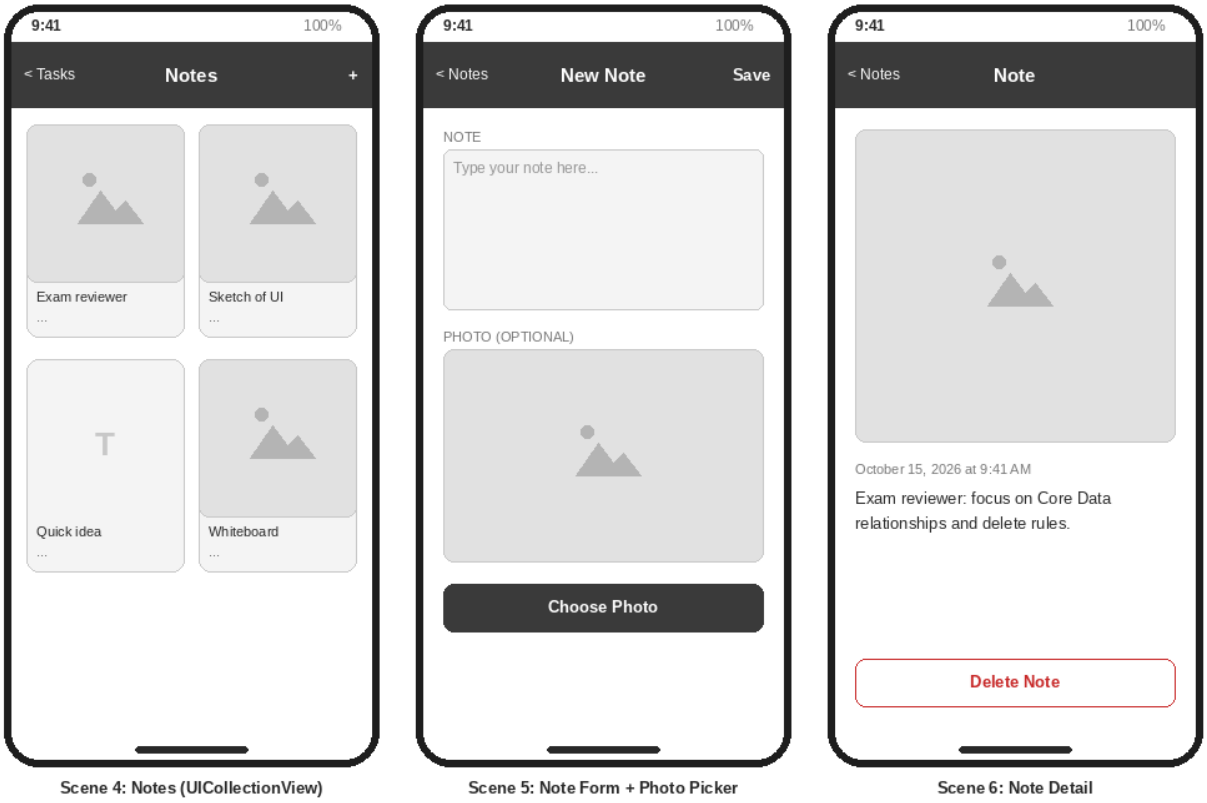


Figure 2. Notes grid (UICollectionView), Note Form with photo picker, and Note Detail screens.

The diagram below shows how the six screens are connected through manual segues, and which object is handed from one screen to the next in prepare(for:sender:). Every screen is pushed onto a single UINavigationController, so each one gets an automatic Back button.

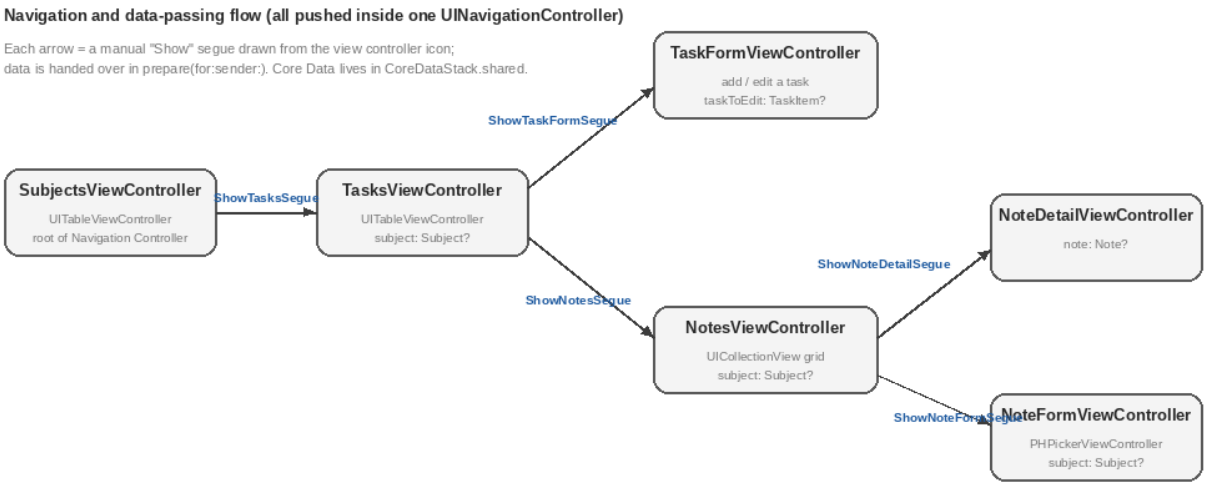


Figure 3. Navigation and data-passing flow between the six view controllers.

Segue identifier	From	To	Triggered by	Data passed
ShowTasksSegue	SubjectsVC	TasksVC	Tap a subject row	subject

Segue identifier	From	To	Triggered by	Data passed
ShowTaskFormSegue	TasksVC	TaskFormVC	+ button (add) or tap a task row (edit)	subject, taskToEdit
ShowNotesSegue	TasksVC	NotesVC	Notes button	subject
ShowNoteFormSegue	NotesVC	NoteFormVC	+ button	subject
ShowNoteDetailSegue	NotesVC	NoteDetailVC	Tap a note cell	note

5. Functional Requirements

Work through Parts A to H **in order**. The steps are numbered continuously from Step 1. Each Part ends with a Git commit, so you can always go back to a working version if something breaks. Menu names below are for Xcode 26; if your version differs slightly, look for the closest menu item.

Before You Begin: Xcode Quick Reference

Term	Where to find it	What it is
Project Navigator	Left panel (⌘1)	The list of all files in your project.
Inspectors	Right panel (⌥⌘0 shows or hides it)	Tabs for File, Identity (class name), Attributes, Size, and Connections of the selected item.
Library	+ button at the top right (⇧⌘L)	Buttons, labels, views, and other items you drag onto the storyboard.
Document Outline	Left side of the storyboard canvas	A tree of every scene and control. Easier to select things in than the canvas.
Assistant Editor	Editor > Assistant	Shows the storyboard and a Swift file side by side, so you can connect controls to code.
Run / Stop	Top left (⌘R / ⌘.)	Builds and launches the app. Stops it.

> **Tip:** If the app crashes at launch with “this class is not key value coding-compliant”, an outlet is connected to something that has been removed or renamed. Select the view controller in the storyboard, open the **Connections Inspector** (the last tab on the right), and remove any connection that shows a yellow warning.

Part A — Project Setup, Git, and the Core Data Model

Goal: Create the Xcode project, start version control, and build the three-entity database (Subject, TaskItem, Note) that the whole app uses.

Step 1 — Create the Xcode project

- Open Xcode and choose **File > New > Project…**. Select the **iOS** tab, choose **App**, and click **Next**.
- Product Name: `StudentOrganizer`. Organization Identifier: `com.yourlastname` (for example `com.delacruz`).
- Interface: **Storyboard**. Language: **Swift**. Storage (or “Use Core Data” in older Xcode): **None / unchecked**. Include Tests: **unchecked**. We will add Core Data ourselves so you can see every piece.
- Click **Next**, choose a folder you can find later (for example Documents), and click **Create**.

> **Watch out:** Do not use SwiftUI. This project is storyboard-based (UIKit). If the Interface menu says SwiftUI, change it to Storyboard before clicking Next.

Step 2 — Start version control (Git)

- Choose **Source Control > Create Git Repositories…**, tick your project, and click **Create**. (If Xcode already created a repository for you, this item is greyed out, which is fine.)
- Open the **Source Control navigator** (the branch icon at the top of the left panel) and confirm you see the **main** branch.

From now on, every Part ends with a commit. A commit is a saved snapshot of your project with a message describing what you did.

Step 3 — Set the minimum iOS version and lock the orientation

- Click the blue **StudentOrganizer** project icon at the top of the Project Navigator, select the **StudentOrganizer** target, and open the **General** tab.
- Under **Minimum Deployments**, set iOS to **16.0**. If your own iPhone runs an older iOS than 16, set it to your iPhone's version instead, but never below **15.0**.
- Under **Supported Destinations**, remove **iPad** if it is listed. Under **Deployment Info**, keep only **Portrait** ticked for iPhone orientation.

> **Tip:** New Xcode versions default to the newest iOS. An iPhone that is on an older iOS cannot install your app unless the minimum version is lowered. Fixing this now saves you trouble in Part H.

Step 4 — Run the empty template once

- In the toolbar at the top, click the device menu and choose an iPhone simulator (for example iPhone 17).
- Press the **Run** button (▶) or ⌘R. After the build finishes you should see the Simulator open with a blank white screen.
- Press **Stop** (■) or ⌘. to stop the app.

> **Check:** If the Simulator shows a blank white screen, your project is healthy. If the build fails, re-read Steps 1–3 before continuing.

Step 5 — Create the Core Data model file

- Choose **File > New > File from Template…**, scroll to the **Core Data** section, choose **Data Model**, and click **Next**.
- Name it exactly `StudentOrganizer` (same as the project) and click **Create**. A file named `StudentOrganizer.xcdatamodeld` appears in the Project Navigator.
- Click this file. The **model editor** opens. This is where you design your database tables, which Core Data calls **entities**.

> **Watch out:** The model name must match the name used in the code later (`NSPersistentContainer(name: "StudentOrganizer")`). A different name causes a crash at launch.

Step 6 — Create the Subject entity

- At the bottom of the model editor click **Add Entity**. Double-click the new entity name, "Entity", and rename it **Subject**.
- With Subject selected, click the **+** under **Attributes** twice and create these attributes: `name` of type **String** and `createdAt` of type **Date**.
- Select each attribute, open the **Data Model Inspector** (right panel, last tab) and **untick Optional** for both. These values are always required.

Step 7 — Create the TaskItem entity

- Click **Add Entity** again and rename it **TaskItem**. Add these attributes: `title` (String, not optional), `details` (String, **Optional ticked**), `dueDate` (Date, not optional), `isDone` (Boolean, not optional, Default Value **NO**), and `priority` (Integer 16, not optional, Default Value **1**).
- Priority will store 0 = Low, 1 = Medium, 2 = High.

> **Watch out:** Name the entity **TaskItem**, not "Task". Swift already has a built-in type called Task, and using the same name causes confusing errors. Also never name an attribute `description`; Core Data reserves it.

Step 8 — Create the Note entity

- Click **Add Entity**, rename it **Note**, and add: `text` (String, not optional), `createdAt` (Date, not optional), and `photoData` (**Binary Data**, **Optional ticked**).
- Select `photoData` and, in the Data Model Inspector, tick **Allows External Storage**. Core Data then keeps large photos in separate files instead of inside the database, which keeps the database fast.

Step 9 — Connect the entities with relationships

A relationship tells Core Data which tasks and notes belong to which subject. We need relationships in both directions.

- Select **Subject**. Under **Relationships** click **+**. Set Name: `tasks`, Destination: **TaskItem**, Type: **To Many**, Delete Rule: **Cascade**.
- Click **+** again. Set Name: `notes`, Destination: **Note**, Type: **To Many**, Delete Rule: **Cascade**.
- Select **TaskItem**. Click **+** under Relationships. Set Name: `subject`, Destination: **Subject**, **Inverse: tasks**, Type: **To One**, Delete Rule: **Nullify**.
- Select **Note**. Click **+**. Set Name: `subject`, Destination: **Subject**, **Inverse: notes**, Type: **To One**, Delete Rule: **Nullify**.
- Switch **Editor Style** (bottom right of the model editor) to **Graph**. You should see three boxes joined by lines. Take a screenshot of this graph; you will submit it.

> **Tip:** Cascade means "when the subject is deleted, delete its tasks and notes too". Nullify means "when a single task is deleted, just remove the link; do not delete the subject".

Entity	Attribute / relationship	Type	Optional?	Notes
Subject	name	String	No	e.g., "Web Systems"
	createdAt	Date	No	
	tasks	To Many → TaskItem	—	Delete rule: Cascade
	notes	To Many → Note	—	Delete rule: Cascade
TaskItem	title	String	No	
	details	String	Yes	
	dueDate	Date	No	
	isDone	Boolean	No	Default: NO
	priority	Integer 16	No	Default: 1 (0 Low, 1 Medium, 2 High)

Entity	Attribute / relationship	Type	Optional?	Notes
	subject	To One → Subject	—	Inverse: tasks; Nullify
Note	text	String	No	
	createdAt	Date	No	
	photoData	Binary Data	Yes	Allows External Storage
	subject	To One → Subject	—	Inverse: notes; Nullify

Step 10 — Let Xcode generate the Swift classes

- Select each entity (Subject, TaskItem, Note) and open the **Data Model Inspector**. Make sure **Codegen** is set to **Class Definition** (this is the default) and Module is **Current Product Module**.
- Press ⌘B (Product > Build). The message “Build Succeeded” should appear.

Xcode now secretly creates the Swift classes `Subject`, `TaskItem`, and `Note` for you. You will use them in code just like any other class, for example `Subject(context: …)` creates a new subject.

> **Tip:** If later steps show “Cannot find type ‘Subject’ in scope”, press ⌘B once so Xcode regenerates the classes.

Step 11 — Create CoreDataStack.swift

- Choose **File > New > File from Template…**, select **Swift File**, name it `CoreDataStack`, and click **Create**.
- Delete everything in the file and type in the code below.

File: CoreDataStack.swift

```
import CoreData

/// One shared object that owns the Core Data "stack" (model + database + context).
/// Every screen talks to the database through CoreDataStack.shared.
final class CoreDataStack {

    static let shared = CoreDataStack()

    /// Loads StudentOrganizer.xcdatamodeld and creates the SQLite file on the device.
    let container: NSPersistentContainer

    /// The "scratchpad" where we create, change, and delete objects before saving.
    var context: NSManagedObjectContext { container.viewContext }

    private init() {
        // The name MUST match the .xcdatamodeld file name exactly.
        container = NSPersistentContainer(name: "StudentOrganizer")
        container.loadPersistentStores { storeDescription, error in
            if let error = error {
                // If the database cannot load, the app cannot work at all.
                fatalError("Core Data failed to load: \(error)")
            }
            print("Database file: \(storeDescription.url?.path ?? "unknown")")
        }
    }

    /// Writes any pending changes to the database file.
    func save() {
        guard context.hasChanges else { return }
```

```
        do {
            try context.save()
        } catch {
            print("Core Data save failed: \(error)")
        }
    }
}
```

How it works:

- `NSPersistentContainer` loads your model and creates the database file on the device.
- `viewContext` is the “scratchpad”: you create, change, or delete objects there, and nothing is permanent until you call `save()`.
- `static let shared` means the whole app uses one single stack: `CoreDataStack.shared`.
- The printed “Database file” path in the Xcode console shows where the data lives on the device.

Step 12 — Create Helpers.swift

- Create another Swift File named `Helpers` and replace its contents with the code below. It holds three small helpers used by several screens: an alert function, a photo-resizing function, and a label for empty lists.

File: Helpers.swift

```
import UIKit

extension UIViewController {
    /// Shows a simple alert with one OK button (used for validation messages).
    func showAlert(title: String, message: String) {
        let alert = UIAlertController(title: title, message: message,
                                      preferredStyle: .alert)
        alert.addAction(UIAlertAction(title: "OK", style: .default))
        present(alert, animated: true)
    }
}

extension UIImage {
    /// Returns a smaller copy whose longest side is at most maxDimension pixels.
    /// Photos from the iPhone camera are huge; shrinking them keeps the database small.
    func resized(maxDimension: CGFloat) -> UIImage {
        let longestSide = max(size.width, size.height)
        guard longestSide > maxDimension else { return self }

        let scale = maxDimension / longestSide
        let newSize = CGSize(width: size.width * scale, height: size.height * scale)

        let format = UIGraphicsImageRendererFormat.default()
        format.scale = 1   // 1 point = 1 pixel, so the pixel size is exactly newSize
        return UIGraphicsImageRenderer(size: newSize, format: format).image { _ in
            self.draw(in: CGRect(origin: .zero, size: newSize))
        }
    }
}

/// A centered gray label shown in a list or grid when there is nothing to display.
func emptyStateLabel(_ message: String) -> UILabel {
    let label = UILabel()
    label.text = message
    label.numberOfLines = 0
    label.textAlignment = .center
    label.textColor = .secondaryLabel
    label.font = .preferredFont(forTextStyle: .body)
    return label
}
```

Press ⌘B. The build should still succeed.

Step 13 — Commit Part A

- Choose **Source Control > Commit…**. Make sure all changed files are ticked.
- Write the message `Part A: project setup and Core Data model` and click **Commit**.

Part B — Navigation Controller and the Subjects List

Goal: Build the first screen: a list of subjects inside a Navigation Controller, saved with Core Data.

Step 14 — Clear the template storyboard

- Click **Main.storyboard** in the Project Navigator. The storyboard canvas opens.
- If the **Document Outline** (left side of the canvas) is hidden, click the small panel icon at the bottom-left of the canvas to show it.
- Click **View Controller Scene**, press **Delete**. Then, in the Project Navigator, right-click **ViewController.swift** and choose **Delete > Move to Trash**.

Step 15 — Add the Subjects scene and embed it in a Navigation Controller

- Click the **+** button at the top right of Xcode (or press ⇧⌘L) to open the **Library**. Search for **Table View Controller** and drag it onto the canvas.
- Select the new scene by clicking its yellow icon, open the **Attributes Inspector** (right panel), and tick **Is Initial View Controller**. An arrow appears on its left.
- With the scene still selected choose **Editor > Embed In > Navigation Controller**. A Navigation Controller appears to the left and the entry arrow now points to it.

> **Tip:** The Navigation Controller gives every screen a navigation bar at the top and an automatic Back button. This is the same idea as the back stack you learned in Mobile Application Development 1.

Step 16 — Create the SubjectsViewController class

- Choose **File > New > File from Template… > Cocoa Touch Class**. Class: `SubjectsViewController`. Subclass of: **UITableViewController**. Untick **Also create XIB file**. Language: **Swift**. Click **Next**, then **Create**.
- In the storyboard, click the yellow icon of the table view scene, open the **Identity Inspector** (third tab), and type `SubjectsViewController` in the **Class** field. Press Return.
- Open **SubjectsViewController.swift**, delete all of its contents, and type in this code.

File: SubjectsViewController.swift

```
import UIKit
import CoreData

class SubjectsViewController: UITableViewController {

    // MARK: - Properties
    private var subjects: [Subject] = []
    private let context = CoreDataStack.shared.context

    // MARK: - View life cycle
    override func viewDidLoad() {
        super.viewDidLoad()
        title = "My Subjects"
    }

    // Runs every time the screen appears, so the list is always up to date
    // (for example, after returning from the Tasks screen).
    override func viewWillAppear(_ animated: Bool) {
```

```
    super.viewWillAppear(animated)
    loadSubjects()
}

// MARK: - Core Data
private func loadSubjects() {
    let request: NSFetchRequest<Subject> = Subject.fetchRequest()
    request.sortDescriptors = [NSSortDescriptor(key: "name", ascending: true)]
    do {
        subjects = try context.fetch(request)
    } catch {
        print("Could not fetch subjects: \(error)")
        subjects = []
    }
    tableView.backgroundView = subjects.isEmpty
        ? emptyStateLabel("No subjects yet.\nTap + to add one.") : nil
    tableView.reloadData()
}

// MARK: - Actions
@IBAction func addSubjectTapped(_ sender: UIBarButtonItem) {
    let alert = UIAlertController(title: "New Subject",
                                  message: "Enter the name of the subject.",
                                  preferredStyle: .alert)
    alert.addTextField { textField in
        textField.placeholder = "e.g., Mobile Application Development 2"
        textField.autocapitalizationType = .words
    }
    alert.addAction(UIAlertAction(title: "Cancel", style: .cancel))
    let saveAction = UIAlertAction(title: "Save", style: .default) {
        [weak self, weak alert] _ in
        // guard: stop quietly if the name is empty (never force-unwrap user input)
        guard let self = self,
              let name = alert?.textFields?.first?.text?
                  .trimmingCharacters(in: .whitespacesAndNewlines),
              !name.isEmpty else { return }

        let subject = Subject(context: self.context)   // create a new row
        subject.name = name
        subject.createdAt = Date()
        CoreDataStack.shared.save()                    // write it to the database
        self.loadSubjects()
    }
    alert.addAction(saveAction)
    present(alert, animated: true)
}

// MARK: - Table view data source
override func tableView(_ tableView: UITableView,
                        numberOfRowsInSection section: Int) -> Int {
    return subjects.count
}

override func tableView(_ tableView: UITableView,
                        cellForRowAt indexPath: IndexPath) -> UITableViewCell {
    let cell = tableView.dequeueReusableCell(withIdentifier: "SubjectCell",
                                             for: indexPath)
    let subject = subjects[indexPath.row]

    // subject.tasks is an NSSet of TaskItem objects (the relationship we made).
    let allTasks = subject.tasks as? Set<TaskItem> ?? []
    let pendingCount = allTasks.filter { !$0.isDone }.count

    var content = cell.defaultContentConfiguration()
    content.text = subject.name ?? "Untitled subject"
    content.secondaryText = "\(pendingCount) pending task(s)"
    cell.contentConfiguration = content
    cell.accessoryType = .disclosureIndicator
    return cell
}

// MARK: - Table view delegate
```

```
    // Swipe left on a row to delete it.
    override func tableView(_ tableView: UITableView,
                            commit editingStyle: UITableViewCell.EditingStyle,
                            forRowAt indexPath: IndexPath) {
        guard editingStyle == .delete else { return }
        context.delete(subjects[indexPath.row])   // cascade: tasks and notes go too
        CoreDataStack.shared.save()
        subjects.remove(at: indexPath.row)
        tableView.deleteRows(at: [indexPath], with: .automatic)
        tableView.backgroundView = subjects.isEmpty
            ? emptyStateLabel("No subjects yet.\nTap + to add one.") : nil
    }
}
```

How it works:

- `loadSubjects()` builds a **fetch request**: it asks Core Data for every `Subject`, sorted by name, and stores the result in the `subjects` array.
- `viewWillAppear` runs every time the screen shows, so the list is refreshed after you come back from another screen.
- The **+** button shows a `UIAlertController` with a text field. The `guard` statement stops quietly if the name is empty, and the text is never force-unwrapped.
- `Subject(context:)` creates a new row; `CoreDataStack.shared.save()` writes it to the database.
- Swipe-to-delete removes the subject; the Cascade rule you set in Part A also removes its tasks and notes.

Step 17 — Configure the prototype cell

- In the Document Outline expand **Subjects Scene > Table View > Table View Cell** and select the **Table View Cell**.
- In the **Attributes Inspector** set **Style** to **Subtitle** and **Identifier** to `SubjectCell` (exactly, with the same capital letters as in the code).

> **Watch out:** A wrong cell identifier is the most common crash in table view projects. The message reads “unable to dequeue a cell with identifier”. Compare what you typed in the storyboard with the code character by character.

Step 18 — Add the + button and connect its action

- Open the Library, search for **Bar Button Item**, and drag it to the **right side of the navigation bar** of the Subjects scene.
- Select it and, in the Attributes Inspector, set **System Item** to **Add**.
- Open the Assistant Editor (Editor > Assistant, or the two-circles icon at the top right of Xcode) so the storyboard is on the left and the Swift file is on the right. Each `@IBOutlet` or `@IBAction` line has a small hollow circle in the left gutter. Click and drag from that circle to the matching control; the circle fills in when the connection is made.
- Drag from the circle next to `@IBAction func addSubjectTapped` to the **+ bar button item**.

Step 19 — Run and test the Subjects list

- Run the app. Tap **+**, type a subject name such as “Mobile Application Development 2”, and tap **Save**. The subject appears in the list.
- Add one more subject, then swipe left on a row to delete it.
- Press **Stop**, then **Run** again. Your subjects are still there. That is data persistence working.

> **Watch out:** Do not tap a subject row yet. The screen it opens does not exist until Part C.

Step 20 — Commit Part B

- Choose **Source Control > Commit…** and use the message `Part B: navigation controller and subjects list`.

Part C — Tasks List and the Task Form

Goal: Open a subject to see its tasks. Add, edit, complete, and delete tasks using a form with text fields, a date picker, a segmented control, and a switch.

Step 21 — Add the Tasks scene and its class

- Drag another **Table View Controller** from the Library onto the canvas, to the right of the Subjects scene.
- Create a new Cocoa Touch Class named `TasksViewController` (subclass of **UITableViewController**, no XIB), as you did in Part B.
- Select the new scene, open the **Identity Inspector**, and set **Class** to `TasksViewController`.
- Replace the contents of **TasksViewController.swift** with this code.

File: TasksViewController.swift

```
import UIKit
import CoreData

class TasksViewController: UITableViewController {

    // MARK: - Properties
    /// Set by SubjectsViewController in prepare(for:sender:).
    var subject: Subject?

    private var tasks: [TaskItem] = []
    private let context = CoreDataStack.shared.context

    // MARK: - View life cycle
    override func viewDidLoad() {
        super.viewDidLoad()
        title = subject?.name
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        loadTasks()   // refresh after returning from the Task Form
    }

    // MARK: - Core Data
    private func loadTasks() {
        guard let subject = subject else { return }
        let request: NSFetchRequest<TaskItem> = TaskItem.fetchRequest()
        // Only the tasks that belong to THIS subject.
        request.predicate = NSPredicate(format: "subject == %@", subject)
        // Unfinished tasks first, then the earliest due date first.
        request.sortDescriptors = [
            NSSortDescriptor(key: "isDone", ascending: true),
            NSSortDescriptor(key: "dueDate", ascending: true)
        ]
        do {
            tasks = try context.fetch(request)
        } catch {
            print("Could not fetch tasks: \(error)")
            tasks = []
        }
        tableView.backgroundView = tasks.isEmpty
            ? emptyStateLabel("No tasks yet.\nTap + to add one.") : nil
        tableView.reloadData()
    }
```

```
// MARK: - Actions
@IBAction func addTaskTapped(_ sender: UIBarButtonItem) {
    // sender: nil means "no existing task", so the form opens in "add" mode.
    performSegue(withIdentifier: "ShowTaskFormSegue", sender: nil)
}

@IBAction func notesTapped(_ sender: UIBarButtonItem) {
    performSegue(withIdentifier: "ShowNotesSegue", sender: nil)
}

// MARK: - Table view data source
override func tableView(_ tableView: UITableView,
                        numberOfRowsInSection section: Int) -> Int {
    return tasks.count
}

override func tableView(_ tableView: UITableView,
                        cellForRowAt indexPath: IndexPath) -> UITableViewCell {
    let cell = tableView.dequeueReusableCell(withIdentifier: "TaskCell",
                                             for: indexPath)
    let task = tasks[indexPath.row]

    let dueText = task.dueDate?.formatted(date: .abbreviated, time: .shortened)
        ?? "no due date"

    var content = cell.defaultContentConfiguration()
    content.text = task.title ?? "Untitled task"
    content.secondaryText = "Due \(dueText) - \(priorityName(for: task.priority))"
    content.textProperties.color = task.isDone ? .secondaryLabel : .label
    cell.contentConfiguration = content
    cell.accessoryType = task.isDone ? .checkmark : .none
    return cell
}

private func priorityName(for value: Int16) -> String {
    switch value {
    case 0:  return "Low"
    case 2:  return "High"
    default: return "Medium"
    }
}

// MARK: - Table view delegate
override func tableView(_ tableView: UITableView,
                        didSelectRowAt indexPath: IndexPath) {
    tableView.deselectRow(at: indexPath, animated: true)
    // Pass the tapped task so the form opens in "edit" mode.
    performSegue(withIdentifier: "ShowTaskFormSegue",
                 sender: tasks[indexPath.row])
}

// Swipe right: mark done / undo.
override func tableView(_ tableView: UITableView,
    leadingSwipeActionsConfigurationForRowAt indexPath: IndexPath)
    -> UISwipeActionsConfiguration? {
    let task = tasks[indexPath.row]
    let title = task.isDone ? "Undo" : "Done"
    let action = UIContextualAction(style: .normal, title: title) {
        [weak self] _, _, completion in
        task.isDone.toggle()
        CoreDataStack.shared.save()
        self?.loadTasks()
        completion(true)
    }
    action.backgroundColor = .systemGreen
    return UISwipeActionsConfiguration(actions: [action])
}

// Swipe left: delete.
override func tableView(_ tableView: UITableView,
```

```
        trailingSwipeActionsConfigurationForRowAt indexPath: IndexPath)
        -> UISwipeActionsConfiguration? {
        let action = UIContextualAction(style: .destructive, title: "Delete") {
            [weak self] _, _, completion in
            guard let self = self else { completion(false); return }
            self.context.delete(self.tasks[indexPath.row])
            CoreDataStack.shared.save()
            self.loadTasks()
            completion(true)
        }
        return UISwipeActionsConfiguration(actions: [action])
    }

    // MARK: - Navigation
    override func prepare(for segue: UIStoryboardSegue, sender: Any?) {
        switch segue.identifier {
        case "ShowTaskFormSegue":
            guard let destination = segue.destination as? TaskFormViewController
            else { return }
            destination.subject = subject
            destination.taskToEdit = sender as? TaskItem   // nil when adding a new task
        default:
            break
        }
    }
}
```

How it works:

- `var subject: Subject?` receives the subject that was tapped on the previous screen. This is how data is passed forward.
- `NSPredicate(format: "subject == %@", subject)` filters the fetch so only this subject's tasks are returned.
- The two sort descriptors put unfinished tasks first and sort by due date.
- Swipe right marks a task done (green action) and swipe left deletes it (`UIContextualAction`).
- Tapping a row passes the task as the `sender`, so the form opens in **edit** mode. The + button passes `nil`, so the form opens in **add** mode.

Step 22 — Set up the Tasks prototype cell

- Select the **Table View Cell** of the Tasks scene. Set **Style** to **Subtitle** and **Identifier** to `TaskCell`.

Step 23 — Draw the segue from Subjects to Tasks

- Click the **yellow view controller icon** at the top of the **Subjects** scene. Hold **Control** and drag from it to the **Tasks** scene. In the pop-up, choose **Show** under **Manual Segue**.
- Click the new arrow on the canvas. In the **Attributes Inspector** set **Identifier** to `ShowTasksSegue`.
- Now open **SubjectsViewController.swift** and paste the code below just **before the final closing brace** (`}`) of the class.

Add to SubjectsViewController.swift

```
    // MARK: - Table view delegate: open the tapped subject
    override func tableView(_ tableView: UITableView,
                            didSelectRowAt indexPath: IndexPath) {
        tableView.deselectRow(at: indexPath, animated: true)
        // sender: the Subject that was tapped. prepare(for:sender:) receives it.
        performSegue(withIdentifier: "ShowTasksSegue",
                     sender: subjects[indexPath.row])
    }

    // MARK: - Navigation
```

```
    override func prepare(for segue: UIStoryboardSegue, sender: Any?) {
        guard segue.identifier == "ShowTasksSegue",
              let destination = segue.destination as? TasksViewController,
              let subject = sender as? Subject else { return }
        destination.subject = subject   // hand the chosen subject to the next screen
    }
```

> **Watch out:** Draw the segue from the yellow icon of the scene (or its entry in the Document Outline), NOT from a table cell. The code decides when to navigate, using `performSegue(withIdentifier:sender:)`.

Step 24 — Add the + and Notes buttons to the Tasks scene

- Drag a **Bar Button Item** to the right side of the Tasks navigation bar and set **System Item** to **Add**.
- Drag a second **Bar Button Item** to the left of the first. In the Attributes Inspector change **Title** to `Notes` and **Style** to **Plain**.
- Open the Assistant Editor (Editor > Assistant, or the two-circles icon at the top right of Xcode) so the storyboard is on the left and the Swift file is on the right. Each `@IBOutlet` or `@IBAction` line has a small hollow circle in the left gutter. Click and drag from that circle to the matching control; the circle fills in when the connection is made.
- Connect `addTaskTapped` to the **+** button and `notesTapped` to the **Notes** button.

Step 25 — Create the TaskFormViewController class and scene

- Drag a **View Controller** from the Library to the right of the Tasks scene.
- Create a new Cocoa Touch Class named `TaskFormViewController`, subclass of **UIViewController**. Set it as the scene's **Class** in the Identity Inspector.
- Replace the contents of the new Swift file with the code below. It will show errors about connections until Step 28 is finished; that is normal.

File: TaskFormViewController.swift

```
import UIKit

class TaskFormViewController: UIViewController {

    // MARK: - Data passed in from TasksViewController
    var subject: Subject?
    var taskToEdit: TaskItem?     // nil = adding a new task, not nil = editing

    // MARK: - Outlets
    @IBOutlet weak var titleTextField: UITextField!
    @IBOutlet weak var detailsTextField: UITextField!
    @IBOutlet weak var dueDatePicker: UIDatePicker!
    @IBOutlet weak var prioritySegmentedControl: UISegmentedControl!
    @IBOutlet weak var completedSwitch: UISwitch!

    // MARK: - View life cycle
    override func viewDidLoad() {
        super.viewDidLoad()

        if let task = taskToEdit {
            // Edit mode: fill the controls with the saved values.
            title = "Edit Task"
            titleTextField.text = task.title
            detailsTextField.text = task.details
            dueDatePicker.date = task.dueDate ?? Date()
            prioritySegmentedControl.selectedSegmentIndex = Int(task.priority)
            completedSwitch.isOn = task.isDone
        } else {
            title = "New Task"
            dueDatePicker.date = Date()
        }
```

```
        // Tap anywhere on the background to hide the keyboard.
        let tap = UITapGestureRecognizer(target: self, action: #selector(dismissKeyboard))
        tap.cancelsTouchesInView = false   // do not block taps on the other controls
        view.addGestureRecognizer(tap)
    }

    @objc private func dismissKeyboard() {
        view.endEditing(true)
    }

    // MARK: - Actions
    @IBAction func saveTapped(_ sender: UIBarButtonItem) {
        view.endEditing(true)

        // Read the text field safely: it is optional, so never force-unwrap it.
        let taskTitle = (titleTextField.text ?? "")
            .trimmingCharacters(in: .whitespacesAndNewlines)
        guard !taskTitle.isEmpty else {
            showAlert(title: "Title required",
                      message: "Please type a title for your task.")
            return
        }

        // Reuse the task being edited, or create a brand-new one.
        let task = taskToEdit ?? TaskItem(context: CoreDataStack.shared.context)
        task.title = taskTitle
        task.details = detailsTextField.text
        task.dueDate = dueDatePicker.date
        task.priority = Int16(prioritySegmentedControl.selectedSegmentIndex)
        task.isDone = completedSwitch.isOn
        if taskToEdit == nil {
            task.subject = subject    // link the new task to its subject
        }

        CoreDataStack.shared.save()
        navigationController?.popViewController(animated: true)
    }
}
```

How it works:

- `taskToEdit` decides the mode: `nil` means a new task, an object means editing that task.
- `titleTextField.text` is an **Optional** String. We read it with `?? ""` instead of force-unwrapping, then check that it is not empty.
- `taskToEdit ?? TaskItem(context:)` reuses the task being edited or creates a new one.
- `view.endEditing(true)` hides the keyboard before saving.

Step 26 — Design the Task Form with a Stack View

A **Stack View** arranges its children in a row or column automatically, so you need very few constraints.

- From the Library, drag a **Vertical Stack View** into the new scene. In the **Attributes Inspector** set Alignment: **Fill**, Distribution: **Fill**, Spacing: **12**.
- With the stack view selected click the **Add New Constraints** button (bottom-right of the canvas). Untick **Constrain to margins**, then set **Top 20**, **Leading 20**, **Trailing 20** (relative to the **Safe Area**) and click **Add 3 Constraints**.
- Drag these controls **into the stack view** (drop them on the stack in the Document Outline), in this order: a **Label** “Title”; a **Text Field** (Placeholder: “Task title”); a **Label** “Details (optional)”; a **Text Field** (Placeholder: “Details”); a **Label** “Due Date”; a **Date Picker**; a **Label** “Priority”; a **Segmented Control**.

- Select the **Date Picker**: Mode **Date and Time**, Preferred Style **Compact**.
- Select the **Segmented Control**: set **Segments** to **3** and rename them **Low**, **Medium**, **High**. Use the **Segment** drop-down to select Segment 1 (Medium) and tick **Selected**.
- Drag a **Horizontal Stack View** into the vertical stack. Put a **Label** "Completed" and a **Switch** inside it (Alignment **Center**). Leave the switch **off**.
- Drag a **Bar Button Item** onto the right side of this scene's navigation bar and set **System Item** to **Save**.

Step 27 — Draw the segue to the Task Form

- Control-drag from the **yellow icon of the Tasks scene** to the **Task Form scene** and choose **Show** (Manual Segue).
- Select the arrow and set **Identifier** to `ShowTaskFormSegue`.

Step 28 — Connect the Task Form outlets and action

- Open the Assistant Editor (Editor > Assistant, or the two-circles icon at the top right of Xcode) so the storyboard is on the left and the Swift file is on the right. Each `@IBOutlet` or `@IBAction` line has a small hollow circle in the left gutter. Click and drag from that circle to the matching control; the circle fills in when the connection is made.

Code	Drag the circle to…
`titleTextField`	the "Task title" Text Field
`detailsTextField`	the "Details" Text Field
`dueDatePicker`	the Date Picker
`prioritySegmentedControl`	the Segmented Control
`completedSwitch`	the Switch
`saveTapped`	the Save bar button item

Step 29 — Run and test the Tasks screens

- Run the app, tap a subject, then tap **+**. Type a title, pick a due date and a priority, and tap **Save**.
- Swipe a task to the right to mark it done (a checkmark appears) and to the left to delete it. Tap a task to edit it.
- Try saving a task with an empty title. An alert should appear and the app must not crash.
- Go back to the Subjects list. The subtitle should show the number of pending tasks.

Step 30 — Commit Part C

- Choose **Source Control > Commit…** with the message `Part C: tasks list and task form`.

Part D — Notes Grid with UICollectionView

Goal: Show each subject's notes in a two-column photo grid using a UICollectionView and a custom cell.

Step 31 — Add the Notes scene and its class

- Drag a **View Controller** from the Library onto the canvas.
- Create a Cocoa Touch Class named `NotesViewController`, subclass of **UIViewController** (not a collection view controller). Set it as the scene's **Class**.
- Replace the contents of the Swift file with this code.

File: NotesViewController.swift

```
import UIKit
import CoreData

class NotesViewController: UIViewController,
                           UICollectionViewDataSource,
                           UICollectionViewDelegateFlowLayout {

    // MARK: - Properties
    var subject: Subject?

    private var notes: [Note] = []
    private let context = CoreDataStack.shared.context

    private let columns: CGFloat = 2    // number of cells per row
    private let spacing: CGFloat = 12   // gap between cells and around the grid

    @IBOutlet weak var collectionView: UICollectionView!

    // MARK: - View life cycle
    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Notes"
        collectionView.dataSource = self
        collectionView.delegate = self
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
        loadNotes()
    }

    // MARK: - Core Data
    private func loadNotes() {
        guard let subject = subject else { return }
        let request: NSFetchRequest<Note> = Note.fetchRequest()
        request.predicate = NSPredicate(format: "subject == %@", subject)
        request.sortDescriptors = [NSSortDescriptor(key: "createdAt", ascending: false)]
        do {
            notes = try context.fetch(request)
        } catch {
            print("Could not fetch notes: \(error)")
            notes = []
        }
        collectionView.backgroundView = notes.isEmpty
            ? emptyStateLabel("No notes yet.\nTap + to add one.") : nil
        collectionView.reloadData()
    }

    // MARK: - Actions
    @IBAction func addNoteTapped(_ sender: UIBarButtonItem) {
        performSegue(withIdentifier: "ShowNoteFormSegue", sender: nil)
    }

    // MARK: - UICollectionViewDataSource
    func collectionView(_ collectionView: UICollectionView,
```

```
                        numberOfItemsInSection section: Int) -> Int {
        return notes.count
    }

    func collectionView(_ collectionView: UICollectionView,
                        cellForItemAt indexPath: IndexPath) -> UICollectionViewCell {
        guard let cell = collectionView.dequeueReusableCell(
                withReuseIdentifier: "NoteCell", for: indexPath) as? NoteCell else {
            return UICollectionViewCell()
        }
        cell.configure(with: notes[indexPath.item])
        return cell
    }

    // MARK: - UICollectionViewDelegate
    func collectionView(_ collectionView: UICollectionView,
                        didSelectItemAt indexPath: IndexPath) {
        performSegue(withIdentifier: "ShowNoteDetailSegue",
                     sender: notes[indexPath.item])
    }

    // MARK: - UICollectionViewDelegateFlowLayout (grid size and spacing)
    func collectionView(_ collectionView: UICollectionView,
                        layout collectionViewLayout: UICollectionViewLayout,
                        sizeForItemAt indexPath: IndexPath) -> CGSize {
        // Total width minus all gaps, divided equally between the columns.
        let totalSpacing = spacing * (columns + 1)
        let width = floor((collectionView.bounds.width - totalSpacing) / columns)
        return CGSize(width: width, height: width + 40)   // square photo + label area
    }

    func collectionView(_ collectionView: UICollectionView,
                        layout collectionViewLayout: UICollectionViewLayout,
                        insetForSectionAt section: Int) -> UIEdgeInsets {
        return UIEdgeInsets(top: spacing, left: spacing, bottom: spacing, right: spacing)
    }

    func collectionView(  collectionView: UICollectionView,
                        layout collectionViewLayout: UICollectionViewLayout,
                        minimumLineSpacingForSectionAt section: Int) -> CGFloat {
        return spacing
    }

    func collectionView(_ collectionView: UICollectionView,
                        layout collectionViewLayout: UICollectionViewLayout,
                        minimumInteritemSpacingForSectionAt section: Int) -> CGFloat {
        return spacing
    }
}
```

How it works:

- The class adopts `UICollectionViewDataSource` (it supplies the cells) and `UICollectionViewDelegateFlowLayout` (it controls sizes and spacing).
- `sizeForItemAt` divides the screen width equally between 2 columns, so the grid fits any iPhone.
- `didSelectItemAt` is the grid's version of `didSelectRowAt`; it opens the note detail screen.

Step 32 — Add the Collection View

- Drag a **Collection View** from the Library into the Notes scene so it fills the whole screen.
- Click **Add New Constraints** and set **Top 0, Leading 0, Trailing 0, Bottom 0** relative to the **Safe Area** (Constrain to margins unticked).
- In the Attributes Inspector set **Background** to **System Background Color**.

- In the Document Outline select **Collection View Flow Layout**. In the **Size Inspector** set **Estimate Size** to **None**. (If you skip this, your size method is ignored.)

> **Watch out:** Make sure you dragged a **Collection View**, not a Collection View Controller. A Collection View Controller would add a second scene.

Step 33 — Create the NoteCell class and design the cell

- Create a Cocoa Touch Class named `NoteCell`, subclass of **UICollectionViewCell**, no XIB. Replace its contents with the code below.

File: NoteCell.swift

```
import UIKit

class NoteCell: UICollectionViewCell {

    @IBOutlet weak var photoImageView: UIImageView!
    @IBOutlet weak var noteLabel: UILabel!

    // Called once when the cell is loaded from the storyboard: set up its look.
    override func awakeFromNib() {
        super.awakeFromNib()
        layer.cornerRadius = 12
        clipsToBounds = true
        backgroundColor = .secondarySystemBackground
        photoImageView.clipsToBounds = true
    }

    /// Fills the cell with one Note from the database.
    func configure(with note: Note) {
        noteLabel.text = note.text

        if let data = note.photoData, let image = UIImage(data: data) {
            photoImageView.image = image
            photoImageView.contentMode = .scaleAspectFill   // fill the square, crop extra
        } else {
            // No photo: show a text icon instead.
            photoImageView.image = UIImage(systemName: "text.alignleft")
            photoImageView.contentMode = .center
            photoImageView.tintColor = .systemGray3
        }
    }

    // Cells are reused while scrolling, so clear the old picture first.
    override func prepareForReuse() {
        super.prepareForReuse()
        photoImageView.image = nil
    }
}
```

- Back in the storyboard select the **Collection View Cell**. In the **Identity Inspector** set **Class** to `NoteCell`. In the **Attributes Inspector** set **Identifier** to `NoteCell`.
- Drag an **Image View** into the cell's **Content View**. Add constraints **Top 0, Leading 0, Trailing 0** (to the Content View). Then, in the Add New Constraints pop-up, tick **Aspect Ratio** and add it. Select the aspect-ratio constraint in the **Size Inspector**, click **Edit**, and set the **Multiplier** to **1:1** so the image is always a square. Set the image view's **Content Mode** to **Aspect Fill** and tick **Clip to Bounds**.
- Drag a **Label** under the image view. Constraints: **Top 4** (to the image view's bottom), **Leading 8, Trailing 8, Bottom 4** (to the Content View). In the Attributes Inspector set **Font** to **Footnote** and **Lines** to **2**.

> **Tip:** If Xcode complains that the constraints are ambiguous, click the yellow arrow at the top-right of the canvas and choose **Update Frames**, or re-check that the label has a Bottom constraint.

Step 34 — Connect the Notes scene

- Open the Assistant Editor (Editor > Assistant, or the two-circles icon at the top right of Xcode) so the storyboard is on the left and the Swift file is on the right. Each `@IBOutlet` or `@IBAction` line has a small hollow circle in the left gutter. Click and drag from that circle to the matching control; the circle fills in when the connection is made.
- With **NotesViewController.swift** open, connect `collectionView` to the **Collection View**.
- Select the **Collection View Cell**; the Assistant Editor shows **NoteCell.swift**. Connect `photoImageView` to the **Image View** and `noteLabel` to the **Label**.
- Drag a **Bar Button Item** onto the right of the Notes navigation bar, set **System Item** to **Add**, and connect `addNoteTapped` to it.

Step 35 — Draw the segue from Tasks to Notes

- Control-drag from the **yellow icon of the Tasks scene** to the **Notes scene**; choose **Show**.
- Select the arrow and set **Identifier** to `ShowNotesSegue`.
- Open **TasksViewController.swift**. Inside `prepare(for:sender:)`, add this new case **above** `default:`:

Add to prepare(for:sender:) in TasksViewController.swift

```
        case "ShowNotesSegue":
            guard let destination = segue.destination as? NotesViewController
            else { return }
            destination.subject = subject
```

Step 36 — Run and test the empty grid

- Run the app, open a subject, and tap **Notes**. You should see the message “No notes yet. Tap + to add one.” Do not tap + yet; the note form is built in Part E.

Step 37 — Commit Part D

- Choose **Source Control > Commit…** with the message `Part D: notes grid with UICollectionView`.

Part E — Note Form and the Photo Picker

Goal: Let the user write a note and attach a photo from the photo library, then save both to Core Data.

Step 38 — Add the Note Form scene and class

- Drag a **View Controller** onto the canvas, to the right of the Notes scene.
- Create a Cocoa Touch Class named `NoteFormViewController` (subclass of **UIViewController**) and set it as the scene’s **Class**.
- Replace the Swift file’s contents with this code. It starts with `import PhotosUI`, the framework that contains the photo picker.

File: NoteFormViewController.swift

```
import UIKit
import PhotosUI

class NoteFormViewController: UIViewController, PHPickerViewControllerDelegate {

    // MARK: - Data passed in from NotesViewController
    var subject: Subject?
```

```
// The photo the student picked (nil = no photo).
private var selectedImage: UIImage?

// MARK: - Outlets
@IBOutlet weak var noteTextView: UITextView!
@IBOutlet weak var photoImageView: UIImageView!

// MARK: - View life cycle
override func viewDidLoad() {
    super.viewDidLoad()
    title = "New Note"

    // Give the text view a visible border.
    noteTextView.layer.borderWidth = 1
    noteTextView.layer.borderColor = UIColor.systemGray4.cgColor
    noteTextView.layer.cornerRadius = 8

    // Start with a gray placeholder icon until a photo is chosen.
    photoImageView.image = UIImage(systemName: "photo")
    photoImageView.tintColor = .systemGray3
    photoImageView.contentMode = .center

    let tap = UITapGestureRecognizer(target: self, action: #selector(dismissKeyboard))
    tap.cancelsTouchesInView = false
    view.addGestureRecognizer(tap)
}

@objc private func dismissKeyboard() {
    view.endEditing(true)
}

// MARK: - Actions
@IBAction func choosePhotoTapped(_ sender: UIButton) {
    var configuration = PHPickerConfiguration()
    configuration.filter = .images        // photos only, no videos
    configuration.selectionLimit = 1      // one photo per note

    let picker = PHPickerViewController(configuration: configuration)
    picker.delegate = self
    present(picker, animated: true)
}

@IBAction func saveTapped(_ sender: UIBarButtonItem) {
    view.endEditing(true)

    let text = (noteTextView.text ?? "")
        .trimmingCharacters(in: .whitespacesAndNewlines)
    guard !text.isEmpty else {
        showAlert(title: "Note is empty",
                  message: "Please write something before saving.")
        return
    }

    let note = Note(context: CoreDataStack.shared.context)
    note.text = text
    note.createdAt = Date()
    // Convert the image to JPEG bytes for the Binary Data attribute.
    // If no photo was chosen, selectedImage is nil, so photoData stays nil too.
    note.photoData = selectedImage?.jpegData(compressionQuality: 0.8)
    note.subject = subject

    CoreDataStack.shared.save()
    navigationController?.popViewController(animated: true)
}

// MARK: - PHPickerViewControllerDelegate
// Called when the student taps a photo, or taps Cancel.
func picker(  picker: PHPickerViewController,
            didFinishPicking results: [PHPickerResult]) {
    picker.dismiss(animated: true)
```

```
        // results is empty when the student cancelled.
        guard let provider = results.first?.itemProvider,
              provider.canLoadObject(ofClass: UIImage.self) else { return }

        // Loading happens on a background thread, so update the UI on the main thread.
        provider.loadObject(ofClass: UIImage.self) { [weak self] object, error in
            guard let image = object as? UIImage else { return }
            let smallImage = image.resized(maxDimension: 1024)   // keep the database small
            DispatchQueue.main.async {
                self?.selectedImage = smallImage
                self?.photoImageView.image = smallImage
                self?.photoImageView.contentMode = .scaleAspectFit
            }
        }
    }
}
```

How it works:

- `PHPickerViewController` is Apple's built-in photo picker. It runs outside your app, so the user's whole library stays private and **no permission pop-up or Info.plist entry is needed**.
- `configuration.selectionLimit = 1` allows one photo; `.filter = .images` hides videos.
- The delegate method `didFinishPicking` receives the chosen photo. If the user cancels, the results array is empty and we simply return.
- The image loads on a background thread, so `DispatchQueue.main.async` is used to update the screen.
- `image.resized(maxDimension: 1024)` shrinks the photo, and `jpegData(compressionQuality: 0.8)` converts it to bytes for the `photoData` attribute.

Step 39 — Design the Note Form

- Drag a **Vertical Stack View** into the scene and constrain it like in Part C: **Top 20, Leading 20, Trailing 20** to the Safe Area. Set Spacing to **12**.
- Drag into the stack, in order: a **Label** "Note"; a **Text View**; a **Label** "Photo (optional)"; an **Image View**; a **Button**.
- Select the **Text View** and add a **Height** constraint of **150**. In the Attributes Inspector, **delete the default "Lorem ipsum" text** from the Text field so the box starts empty.
- Select the **Image View**, add a **Height** constraint of **200**, and set **Content Mode** to **Aspect Fit**.
- Select the **Button**. Set **Style** to **Filled** and **Title** to "Choose Photo".
- Drag a **Bar Button Item** to the right of this scene's navigation bar and set **System Item** to **Save**.

Step 40 — Draw the segue and connect the form

- Control-drag from the **yellow icon of the Notes scene** to the **Note Form scene**, choose **Show**, and set the arrow's **Identifier** to `ShowNoteFormSegue`.
- Open **NotesViewController.swift** and paste the code below just before the final closing brace (`}`) of the class.

Add to NotesViewController.swift

```
    // MARK: - Navigation
    override func prepare(for segue: UIStoryboardSegue, sender: Any?) {
        switch segue.identifier {
        case "ShowNoteFormSegue":
            guard let destination = segue.destination as? NoteFormViewController
            else { return }
```

```
            destination.subject = subject
        default:
            break
        }
    }
```

- Open the Assistant Editor (Editor > Assistant, or the two-circles icon at the top right of Xcode) so the storyboard is on the left and the Swift file is on the right. Each `@IBOutlet` or `@IBAction` line has a small hollow circle in the left gutter. Click and drag from that circle to the matching control; the circle fills in when the connection is made.
- In **NoteFormViewController.swift** connect `noteTextView` to the Text View, `photoImageView` to the Image View, `choosePhotoTapped` to the **Choose Photo** button, and `saveTapped` to the **Save** bar button item.

Step 41 — Run and test notes with photos

- Run the app, open a subject, tap **Notes > +**, type a note, tap **Choose Photo**, and pick a picture. It appears in the preview. Tap **Save**.
- The note appears in the grid. Add a second note **without** a photo; it shows a text icon instead.
- Try saving an empty note. An alert should appear.

> **Tip:** The Simulator's photo library may be empty. Drag a picture file from your Mac onto the Simulator window to add it to Photos.

Step 42 — Commit Part E

- Choose **Source Control > Commit…** with the message `Part E: note form with photo picker`.

Part F — Note Detail and Delete

Goal: Tap a note in the grid to view it full-size and delete it after a confirmation.

Step 43 — Add the Note Detail scene and class

- Drag a **View Controller** onto the canvas, to the right of the Notes scene.
- Create a Cocoa Touch Class named `NoteDetailViewController` (**UIViewController**) and set it as the scene's **Class**.
- Replace the Swift file's contents with this code.

File: NoteDetailViewController.swift

```
import UIKit

class NoteDetailViewController: UIViewController {

    // MARK: - Data passed in from NotesViewController
    var note: Note?

    // MARK: - Outlets
    @IBOutlet weak var photoImageView: UIImageView!
    @IBOutlet weak var dateLabel: UILabel!
    @IBOutlet weak var noteLabel: UILabel!

    // MARK: - View life cycle
    override func viewDidLoad() {
        super.viewDidLoad()
        title = "Note"

        guard let note = note else { return }
        noteLabel.text = note.text
```

```
        dateLabel.text = note.createdAt?.formatted(date: .long, time: .shortened)

        if let data = note.photoData, let image = UIImage(data: data) {
            photoImageView.image = image
            photoImageView.contentMode = .scaleAspectFit
        } else {
            photoImageView.image = UIImage(systemName: "text.alignleft")
            photoImageView.contentMode = .center
            photoImageView.tintColor = .systemGray3
        }
    }

    // MARK: - Actions
    @IBAction func deleteTapped(_ sender: UIButton) {
        // Ask first: deleting cannot be undone.
        let alert = UIAlertController(title: "Delete this note?",
                                      message: "This cannot be undone.",
                                      preferredStyle: .alert)
        alert.addAction(UIAlertAction(title: "Cancel", style: .cancel))
        let deleteAction = UIAlertAction(title: "Delete", style: .destructive) {
            [weak self] _ in
            guard let self = self, let note = self.note else { return }
            CoreDataStack.shared.context.delete(note)
            CoreDataStack.shared.save()
            self.navigationController?.popViewController(animated: true)
        }
        alert.addAction(deleteAction)
        present(alert, animated: true)
    }
}
```

How it works: the screen reads the `note` it was given, shows its photo (or an icon), date, and text, and asks for confirmation with an alert before deleting.

Step 44 — Design the Note Detail screen

- Drag a **Vertical Stack View** into the scene, constrained **Top 20, Leading 20, Trailing 20** to the Safe Area, Spacing **12**.
- Drag into the stack, in order: an **Image View** (Height constraint **300**, Content Mode **Aspect Fit**); a **Label** for the date (Font **Footnote**, Color **Secondary Label**); a **Label** for the note text (Lines **0**, so it can wrap); and a **Button** titled “Delete Note” (Style **Plain**, text color **Red**).
- Select the stack view and confirm **Alignment** is **Fill**, so the controls stretch across the width while the stack stays at the top of the screen.

Step 45 — Draw the segue and connect the screen

- Control-drag from the **yellow icon of the Notes scene** to the **Note Detail scene**, choose **Show**, and set the **Identifier** to `ShowNoteDetailSegue`.
- Open **NotesViewController.swift**. Inside `prepare(for:sender:)`, add this case **above** `default:`:

Add to prepare(for:sender:) in NotesViewController.swift

```
        case "ShowNoteDetailSegue":
            guard let destination = segue.destination as? NoteDetailViewController,
                  let note = sender as? Note else { return }
            destination.note = note
```

- Open the Assistant Editor (Editor > Assistant, or the two-circles icon at the top right of Xcode) so the storyboard is on the left and the Swift file is on the right. Each `@IBOutlet` or `@IBAction` line has a small hollow circle in the left gutter. Click and drag from that circle to the matching control; the circle fills in when the connection is made.

- In **NoteDetailViewController.swift** connect `photoImageView` to the Image View, `dateLabel` to the date label, `noteLabel` to the note label, and `deleteTapped` to the **Delete Note** button.

Step 46 — Run and test

- Run the app, open Notes, and tap a note in the grid. The detail screen shows the full photo. Tap **Delete Note**, confirm, and the note disappears from the grid.

Step 47 — Commit Part F

- Choose **Source Control > Commit…** with the message `Part F: note detail and delete`.

Part G — Testing and Cleanup in the Simulator

Goal: Prove that every feature works, that data survives relaunch, and that the layout fits small and large screens. Then document your code.

Step 48 — Run the full test plan

Test each case below in the Simulator and tick it off. Fix any case that fails before continuing.

#	What to do	Expected result
1	Add a subject	It appears in the list, sorted by name.
2	Tap Save on the subject alert with an empty name	Nothing is added and the app does not crash.
3	Add a task with an empty title	An alert says the title is required.
4	Add a task with a due date and High priority	The row shows the due date and “High”.
5	Swipe a task right, then right again	Checkmark and moves to the bottom; second swipe (Undo) restores it.
6	Tap a task, change the title, Save	The list shows the updated title.
7	Swipe a task left	The task is deleted.
8	Add a note with text only	A cell with a text icon appears in the grid.
9	Add a note with a photo	The photo appears in the grid and full-size in the detail screen.
10	Delete a note from its detail screen	A confirmation appears; the note disappears.
11	Stop and re-run the app	All subjects, tasks, and notes are still there.
12	Delete a subject that has tasks and notes	Its tasks and notes are gone; other subjects are unaffected.

Step 49 — Check the layout on two screen sizes

- Run the app on the **smallest** iPhone simulator and on the **largest** iPhone simulator available in your Xcode (use the device menu in the toolbar).
- On both, open the Task Form and the Notes grid. Nothing may be clipped, overlapping, or cut off.
- Take one screenshot from each simulator (⌘S in the Simulator, or File > Save Screen). You will submit them.

Step 50 — Document your code

- Review every file. Add comments that explain **why** the code exists, not just what it says. Example of a good comment: `// Cascade rule also deletes this subject's tasks`. Example of a weak comment: `// delete the subject`.
- Press ⌘B and open the **Issues navigator** (triangle icon). There should be **no warnings or errors**.
- Remove any unused code or leftover test code.

Step 51 — Commit Part G

- Choose **Source Control > Commit…** with the message `Part G: testing and documentation`.

Part H — Run the App on Your Own iPhone

Goal: Install the finished app on a real iPhone using a free Apple ID, and prove that your data persists on the device.

Step 52 — Check what you need

You need	Details
A Mac with Xcode	Xcode must be able to run on your macOS version. For example, Xcode 26 requires macOS Sequoia 15.6 or later. Check Apple's page "Xcode system requirements" if unsure.
An iPhone	Its iOS version must be supported by your Xcode. Your **Minimum Deployments** setting (Step 3) must not be higher than the iPhone's iOS version.
A USB cable	To connect the iPhone to the Mac. Keep the iPhone unlocked while connecting.
An Apple ID	A **free** Apple ID is enough. You do **not** need the paid Apple Developer Program for this project.
Your own Bundle Identifier	Every student must use a unique one, for example `com.lastname.StudentOrganizer`.

> **Tip:** No iPhone? Run the app on the Simulator and tell your instructor so that an alternative can be arranged for the on-device requirement.

Step 53 — Sign in to Xcode with your Apple ID

- Choose **Xcode > Settings… > Accounts**. Click the **+** button, choose **Apple ID**, and sign in.
- After signing in, your name appears with a **Personal Team** entry.

Step 54 — Set up code signing for your project

- Click the blue project icon, select the **StudentOrganizer** target, and open **Signing & Capabilities**.
- Tick **Automatically manage signing**. For **Team**, choose your **Personal Team**.
- Change the **Bundle Identifier** to something unique, such as `com.lastname.StudentOrganizer`. If Xcode says it "failed to register the bundle identifier", someone has used it; add more letters or numbers to make it unique.

Step 55 — Connect and trust your iPhone

- Plug your iPhone into the Mac and unlock it.

- On the iPhone tap **Trust This Computer** and enter your passcode. On Apple-silicon Macs you may also need to click **Allow** on a "connect accessory" message.
- In Xcode choose **Window > Devices and Simulators** (newer Xcode versions call this **Device Hub**). Your iPhone should be listed. Wait until Xcode finishes preparing the device.

Step 56 — Turn on Developer Mode on the iPhone

- On the iPhone open **Settings > Privacy & Security**, scroll to the bottom, and tap **Developer Mode**. Turn it **On**.
- Tap **Restart**. After the iPhone restarts, an alert asks you to confirm. Tap **Turn On** (or **Enable**) and enter your passcode.

> **Tip:** The Developer Mode option appears only after the iPhone has been connected to Xcode. If you cannot find it, reconnect the iPhone, wait for Xcode to recognise it, and look again.

Step 57 — Run the app on the iPhone

- In the Xcode toolbar open the device menu and choose **your iPhone** (it appears under "iOS Devices" or by its name).
- Press **Run** (⌘R). The first build to a new device takes longer than usual.
- Xcode installs the app and tries to launch it.

Step 58 — Trust your developer profile

The first launch usually fails with an "Untrusted Developer" message. This is normal for free accounts.

- On the iPhone open **Settings > General > VPN & Device Management**.
- Under **Developer App**, tap your Apple ID, then tap **Trust** and confirm.
- Press **Run** in Xcode again, or tap the app icon on the iPhone.

Step 59 — Test on the device and capture evidence

- Add at least **2 subjects, 3 tasks (one completed), and 2 notes (one with a photo from your real photo library)**.
- Swipe up to close the app completely, then open it again. Your data must still be there. Take a screenshot **before** closing it and another **after** reopening it.
- Take screenshots of each screen on the iPhone (press the **side button and volume up button** together). Send them to your Mac by AirDrop or cable.
- Take a screenshot of the Xcode window showing **your iPhone's name** in the run-destination menu.

> **Watch out:** Do not delete the app from your iPhone before your demo or defense. Deleting the app also deletes its Core Data database.

Step 60 — Know the limits of a free account

- An app installed with a free Apple ID **stops opening after about a week**. To use it again, connect the iPhone and press Run in Xcode.
- Free accounts cannot use some Apple services (for example push notifications and in-app purchases). This project does not need them. It uses only **local Core Data**, with no iCloud sync.
- To publish on the App Store or share through TestFlight you would need the paid Apple Developer Program, which is not required here.

Step 61 — Troubleshooting

Problem	Likely cause	What to do
Developer Mode is missing in Settings	The iPhone has not been connected to Xcode yet	Connect it, unlock it, wait for Xcode to detect it, then look again.
"Signing requires a development team"	No Team selected	Choose your Personal Team in Signing & Capabilities (Step 54).
"Failed to register bundle identifier"	Identifier already used	Change it to something unique.
iPhone is greyed out or "iOS version not supported"	Minimum Deployments is higher than the iPhone's iOS, or Xcode is too old	Lower Minimum Deployments (not below 15.0) or update Xcode/iOS.
"Untrusted Developer"	Profile not trusted yet	Follow Step 58 (Settings > General > VPN & Device Management).
"Could not launch…device is locked"	The iPhone screen is locked	Unlock the iPhone and press Run again.
iPhone not detected	Cable, port, or trust prompt	Try another cable or port, unlock the iPhone, and tap Trust.
App no longer opens after a week	Free-account signing expired	Press Run in Xcode again with the iPhone connected.

Step 62 — Commit Part H

- Make a final commit with the message `Part H: tested on a physical iPhone`. Then close Xcode and zip the whole project folder (it includes the hidden .git folder).

6. Deliverables

- The complete StudentOrganizer Xcode project folder (including the .git history, with at least one commit per Part, A to H), zipped.
- A screenshot of the Core Data model editor in **Graph** style showing the three entities and their relationships (Part A).
- Screenshots from **your iPhone** (or the Simulator if you were approved to use it): the Subjects list with at least 2 subjects; the Tasks list with at least 3 tasks, one of them completed; the Task Form; the Notes grid with at least 2 notes, one with a photo; and the Note Detail screen.
- Two screenshots proving data persistence on the iPhone: one taken **before** closing the app and one taken **after** reopening it, showing the same data (Part H).
- A screenshot of Xcode showing your iPhone's name as the run destination (Part H).
- A screenshot demonstrating the layout on two different simulator device sizes, the smallest and the largest available (Part G).
- A brief written reflection (3–5 sentences) comparing Core Data and the manual segue / prepare(for:sender:) data passing used here to the local-storage approach and the Intent-based navigation with putExtra()/getStringExtra() pattern from Mobile Application Development 1.

7. Grading Rubric (100 points)

Criterion (100 pts)	Excellent	Good	Fair	Poor
1. Application Functionality & Execution (30 pts)	27–30 – Runs successfully on the Simulator and on a physical iPhone. All required features work (subjects, tasks, notes, photos), data persists after relaunch, and there are no major errors.	23–26 – Runs successfully with minor errors or issues (for example, a list that does not refresh or one missing validation).	18–22 – Runs with several incomplete features or noticeable errors (for example, data lost after relaunch or photos not saved).	12–17 – Does not run properly or has major non-functional features.
2. User Interface & User Experience (20 pts)	18–20 – Clear, consistent, and user-friendly interface with proper navigation and constraints. No clipping on small and large screens.	15–17 – Generally usable with minor design or navigation issues.	12–14 – Usable but has noticeable layout or navigation problems.	8–11 – Difficult to use or poorly organized interface.
3. Code Quality, Documentation & Understanding (20 pts)	18–20 – Well-organized code with meaningful comments; student clearly explains the implementation, including the Core Data model.	15–17 – Organized code with adequate comments; student explains most of the implementation.	12–14 – Limited comments or organization; student explains only some of the code.	8–11 – Poorly organized or undocumented code; student cannot adequately explain the implementation.
4. Completeness & Requirements Compliance (15 pts)	14–15 – All required features and components are complete and follow specifications: all five topics, six screens, and the full data model.	11–13 – Most requirements are complete with minor omissions.	8–10 – Several requirements are incomplete or missing.	5–7 – Many requirements are missing or not properly implemented.
5. Submission, Delivery & Compliance (15 pts)	14–15 – Complete submission (project, screenshots, reflection) is submitted **on or before the deadline** and follows all instructions.	11–13 – Complete submission is **within 1 day late** with minor submission issues.	8–10 – Submission is **2–3 days late** and/or has missing components.	5–7 – Submission is **more than 3 days late** or substantially incomplete.

Code Documentation & Understanding Requirement

Students must include meaningful comments/documentation for important parts of their source code. Comments should explain the purpose or logic of the code, not simply restate the code. The instructor may ask students to explain selected portions of their code to verify their understanding of the submitted implementation.

Code Verification Rule

Students may be asked to explain or demonstrate selected portions of their submitted source code, for example how a fetch request filters tasks by subject, or how the selected photo ends up in the database. Failure to reasonably explain significant portions of the implementation may result in a lower rating under Code Quality, Documentation & Understanding.

8. Submission Instructions

- Submit the zipped project, and the provided project documentation template through the course LMS assignment link before the posted deadline.
- File name format for both the documentation and the zipped project folder: **Group_Name_StudentOrganizer**.
- Put your project's Git repository link in the Web URL submission for this project, you can also put it in the submission comment if some problem occurs.
- Late submissions will not be accepted.

9. Academic Integrity and AI Use

You may use AI tools to clarify a concept (for example, why UITextField.text is Optional, or when to use a fetch request instead of a relationship) or to help interpret an Xcode error message. You must be able to explain every line of your own submission, and any AI-suggested code must be verified by running it in your own project rather than copied without understanding. Submitting another student's project, or a copy of the sample code with only the names changed, is considered a failure to meet the assignment requirements and will not receive credit. **IMPORTANT: paste the share link of your full conversion thread with your AI for checking.**